



武汉理工大学

WUHAN UNIVERSITY OF TECHNOLOGY

企业级应用软件设计与开发课程 大作业报告

项目名称：书灵--AI Agent 书籍推荐系统

方向：方向一：Agentic AI 原生开发

姓名：梁自顺

学号：2025308026

专业：软件工程

指导教师：戚欣

提交日期：2026 年 6 月 22 日

目 录

一， 选题背景与设计思想

- 1.1 问题定义
- 1.2 现有方案不足
- 1.3 项目价值
- 1.4 技术路线

二， Specs 规格文档

- 2.1 Product Spec
- 2.2 Architecture Spec
- 2.3 API Spec

三， 系统架构与设计

- 3.1 总体架构图
- 3.2 Agent 交互流程
- 3.3 数据流设计

四， 关键实现与代码展示

- 4.1 Agent 核心循环
- 4.2 Function Calling 工具定义
- 4.3 LangGraph 工作流实现
- 4.4 配置文件

五， 测试与评估

- 5.1 功能测试
- 5.2 Agent 行为评估
- 5.3 Demo 演示

六， 系统升级与扩展

- 6.1 可扩展架构
- 6.2 下一阶段计划
- 6.3 AI 能力演进路径

七, 课程总结

7.1 个人收获

7.2 工程思维转变

7.3 对课程的建议

一，选题背景与设计思想

1.1 问题定义

在信息过载的时代，读者面临「选书难」的核心痛点。传统书籍推荐方式存在以下问题：用户在海量书籍中难以精准定位符合个人口味的读物；电商平台的推荐算法依赖购买历史，缺乏对读者深层阅读偏好的理解；图书馆和书店的分类体系无法满足个性化需求。

本项目旨在构建一个基于 AI Agent 的智能书籍推荐系统 —— 「书灵」，通过自然语言多轮对话理解用户的阅读偏好，结合 Open Library 真实书籍数据库，提供个性化、有依据的书籍推荐。用户可以通过关键词搜索找到想看的书，也可以与 AI 交流来了解自己的阅读特点，系统支持多轮对话交互，具备长期记忆能力。

1.2 现有方案不足

当前市场上的书籍推荐方案主要包括三类：(1) 电商平台协同过滤推荐(如亚马逊，豆瓣)，这类方案依赖大规模用户行为数据，冷启动问题严重，且推荐理由缺乏可解释性；(2) 基于规则的专家系统，维护成本高，覆盖面窄，无法处理非结构化需求；(3) 通用大语言模型(如 ChatGPT)直接推荐，虽然对话体验自然，但推荐结果往往缺乏真实数据支撑，可能出现「幻觉」书籍(模型虚构不存在书目)。

本系统结合 LLM 的自然语言理解能力与 Open Library 真实书籍 API 的数据可靠性，填补了这一空白 —— 通过 Function Calling 机制让 LLM 有权访问真实书籍数据，确保每次推荐都有据可查。

1.3 项目价值

学术价值：

本项目完整实践了 Agentic AI 的六大核心技术要素 —— SDD 规格驱动开发，Function Calling 工具使用，向量记忆机制(ChromaDB)，多步推理状态机(LangGraph)，多智能体协作(RecommendAgent + SearchAgent)，可观测性与评估(Tracing + Benchmark + LLM0ps)。为 AI Agent 系统工程化提供了可复用的参考实现。

应用价值：

系统采用免费公开的 Open Library API 作为数据源，无需自建书籍数据库即可提供真实推荐；前端采用响应式设计，适配桌面和移动端；所有组件均可本地部署，零外部服务依赖(除 LLM API)。用户可直接在浏览器中与 AI 对话获取推荐。

1.4 技术路线

本项目采用 SDD (Specification-Driven Development, 规格驱动开发) 方法论, 先定义规格文档再编码实现, 确保设计与实现的一致性. 技术路线如下:

阶段	内容	核心产出
SDD 规格	系统概述, API 接口, 数据模型, Agent 行为定义	4 份 Spec 文档
工具层	Open Library 搜索, 书籍详情, 偏好分析	3 个 FC 工具
记忆系统	短期会话记忆 + ChromaDB 长期向量记忆	双层记忆架构
Agent & 工作流	双 Agent 协作 + LangGraph 五节点状态机	多步推理引擎
API & 前端	FastAPI + SSE 流式 + 响应式 Web	端到端可交互系统
测试 & 可观测	26 测试用例 + JSON 日志 + Tracing	质量保障体系

二, Specs 规格文档

2.1 Product Spec (产品规格)

系统概述文件 (spec/01-system-overview.md) 定义了四项核心功能:

- 智能对话推荐: 用户与 AI Agent 进行自然语言多轮对话, AI 通过提问了解阅读偏好 (题材, 难度, 长度, 语言), 基于对话上下文理解用户意图.
- 关键词书籍搜索: 用户输入关键词 (书名, 作者, 题材), 系统通过 Open Library API 检索真实书籍数据, 返回封面, 作者, 简介, 出版信息.
- 偏好分析与记忆: 短期记忆保存当前会话的对话历史 (滑动窗口 + 自动摘要), 长期记忆通过 ChromaDB 向量存储用户偏好, 支持跨会话追踪.
- 多步骤推理推荐: LangGraph 驱动的推荐工作流: 理解意图 -> 搜索书籍 -> 分析偏好 -> 生成推荐, 信息不足时主动追问用户.

2.2 Architecture Spec (架构规格)

Agent 行为规格文件 (spec/04-agent-behaviors.md) 定义了双 Agent 协作体系:

- RecommendAgent (推荐代理): 理解用户意图, 分析阅读偏好, 整合搜索结果生成推荐. 当信息不足时生成追问而非猜测, 确保推荐质量.
- SearchAgent (搜索代理): 调用 Open Library API 执行搜索, 按多维度过滤 (作者, 题材, 年份, 语言), 对结果排序去重.

Agent 之间通过 LangGraph 工作流编排协作, 共享状态 (AgentState TypedDict), 实现松耦合的协作模式.

Function Calling 工具定义 (共 3 个):

工具名	功能描述	关键参数
search_books	关键词搜索书籍, 返回匹配列表	query, limit, subject, language

get_book_detail	获取单本书详细信息	open_library_key
analyze_preferences	分析并保存用户阅读偏好	user_id, summary, genres, authors

2.3 API Spec（接口规格）

接口规格文件 (spec/02-api-spec.md) 定义了完整的 REST API，包含请求/响应格式，SSE 事件类型，错误码等。以下为核心接口一览：

方法	路径	说明	响应类型
POST	/api/chat	发送消息给 AI Agent	SSE 流
POST	/api/search	直接搜索书籍	JSON
GET	/api/conversations	对话列表	JSON
GET	/api/conversations/{id}	对话历史	JSON
DELETE	/api/conversations/{id}	删除对话	JSON
POST	/api/workflow/recommend	LangGraph 推荐	JSON
GET	/api/health	健康检查	JSON

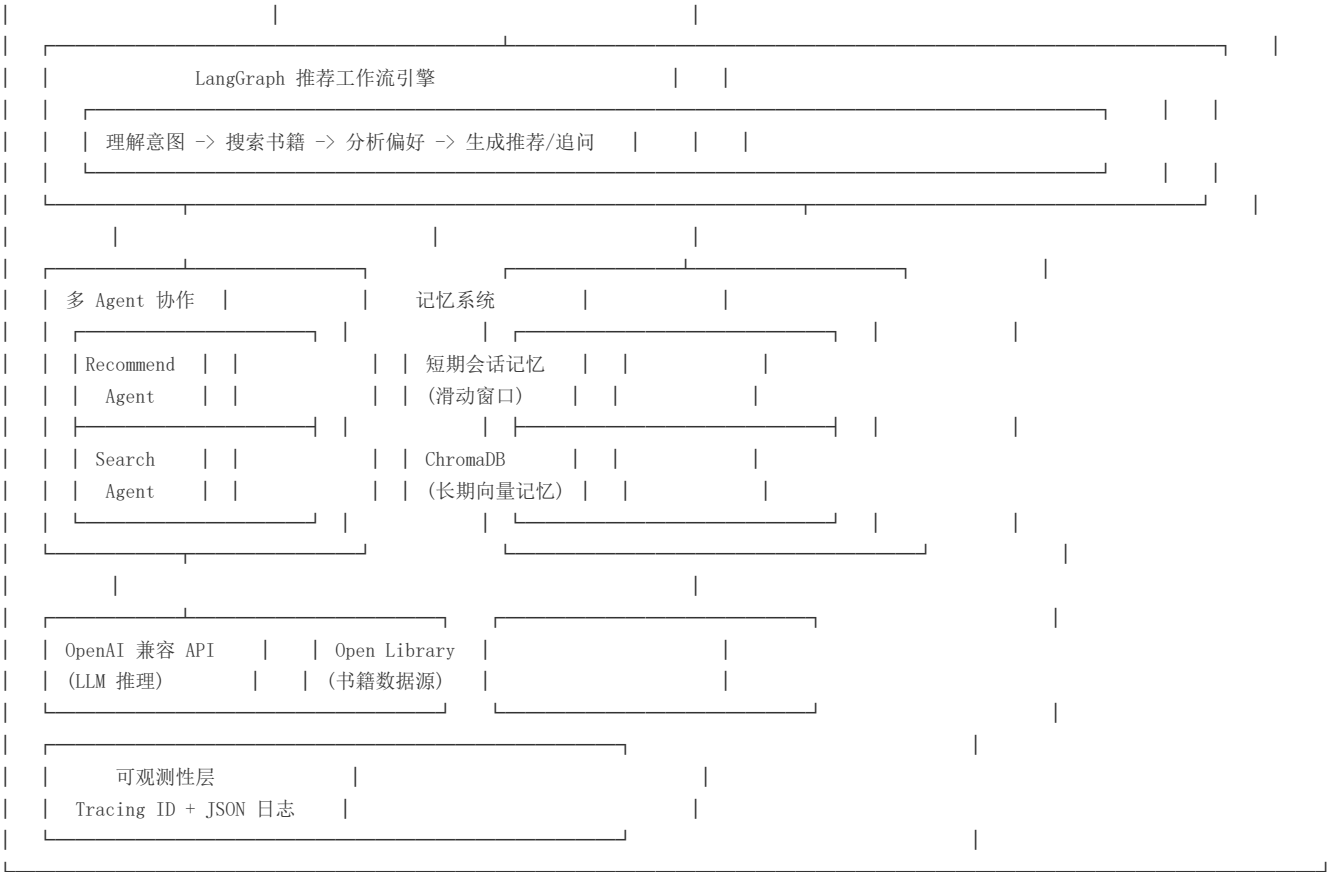
数据模型文件 (spec/03-data-models.md) 使用 Pydantic 定义了 Book, Message, Conversation, UserPreference, AgentState 等核心实体，这些模型同时作为可执行规格 — 编码时 Pydantic 会自动校验数据是否符合规格定义。

三，系统架构与设计

3.1 总体架构图

系统采用经典的分层架构：表示层(响应式 Web 前端) -> 应用层(FastAPI + Agent 系统) -> 基础设施层(LLM API + Open Library + ChromaDB).

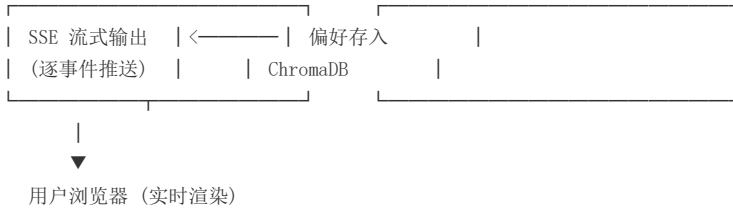




3.2 Agent 交互流程

以下展示用户消息的完整处理流程，从接收消息到 SSE 流式返回结果：





3.3 数据流设计

(1) 对话数据流:

用户输入 -> ConversationMemory (短期存储, 滑动窗口管理) -> LLM 上下文组装 -> Agent 多轮推理 -> 助手回复 -> 回写 ConversationMemory. 超过阈值(20轮)自动压缩, 保留最近对话 + 早期关键上下文.

(2) 偏好数据流:

对话内容 -> AnalyzePreferences 工具(LLM 自主调用) -> 文本向量化(ChromaDB 内置 embedding) -> ChromaDB Collection 存储 -> 下次对话时语义检索偏好 -> 注入 LLM System Prompt 上下文.

(3) 搜索数据流:

Agent Tool Call -> httpx 异步请求 Open Library Search API -> JSON 解析 -> Book Pydantic 模型校验 -> 格式化(format_books_for_llm) -> 返回 LLM 上下文 -> LLM 筛选并生成推荐理由.

四, 关键实现与代码展示

4.1 Agent 核心循环

以下是 Agent 核心推理循环的完整实现 -- 这是 AI Agent 区别于普通 LLM 调用的关键所在. LLM 在每轮对话中自主决定是否调用工具, 调用哪个工具, 传入什么参数(ReAct 模式). 工具结果回传后继续推理, 直到生成最终回复(最多 5 轮循环).


```

async def chat_with_tools(self, messages, tool_handler, system_prompt, max_turns=5):
    full_messages = [{"role": "system", "content": system_prompt}] + messages
    turn = 0
    while turn < max_turns:
        turn += 1
        response = await self._client.chat.completions.create(
            model=self._model, messages=full_messages,
            tools=TOOL_DEFINITIONS,      # 3 个 Function Calling 工具
            tool_choice="auto",          # LLM 自主决定是否调用
            max_tokens=2000,
        )
        msg = response.choices[0].message
        if msg.tool_calls:                # LLM 决定调用工具
            full_messages.append(assistant_msg_with_tool_calls)
            for tc in msg.tool_calls:
                args = json.loads(tc.function.arguments)
                result = await tool_handler(tc.function.name, args)
                full_messages.append({"role": "tool",
                    "tool_call_id": tc.id,
                    "content": json.dumps(result, ensure_ascii=False)})
            yield {"type": "thinking", "content": "正在分析..."}
        else:                             # LLM 生成最终回复
            yield {"type": "message", "content": msg.content}
            yield {"type": "done"}
    return

```

4.2 Function Calling 工具定义

系统定义了 3 个 Function Calling 工具(OpenAI JSON Schema 格式), LLM 根据对话上下文自主选择调用。以下为 `search_books` 工具定义:

```

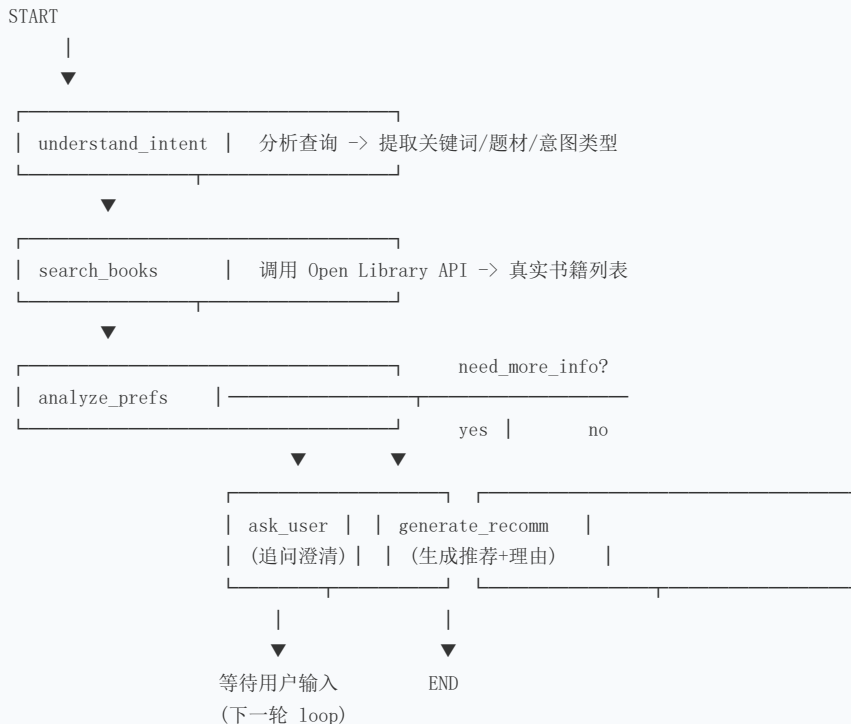
TOOL_DEFINITIONS = [{
    "type": "function",
    "function": {
        "name": "search_books",
        "description": "搜索书籍, 支持书名, 作者, 关键词搜索",
        "parameters": {
            "type": "object",
            "properties": {
                "query": {"type": "string", "description": "搜索关键词"},
                "limit": {"type": "integer", "description": "数量默认10"},
                "subject": {"type": "string", "description": "题材过滤"},
                "language": {"type": "string", "description": "语言过滤 chi/eng"}
            }, "required": ["query"]
        }
    }
}]

# 工具执行: 调用 Open Library 真实 API
async def search_books(query, limit=10, subject=None, language=None):
    params = {"q": query, "limit": limit}
    if subject: params["subject"] = subject
    async with httpx.AsyncClient(timeout=15.0) as client:
        resp = await client.get("https://openlibrary.org/search.json",
                                params=params)
        data = resp.json()
    return [Book(title=doc["title"], ...) for doc in data["docs"]]

```

4.3 LangGraph workflow 实现

系统使用 LangGraph StateGraph 实现多步骤推理状态机, 包含 5 个异步节点和条件路由, 通过 TypedDict 定义共享状态, 编译时注入 MemorySaver 检查点支持持久化.



LangGraph 代码实现

```
workflow = StateGraph(RecommendState)
workflow.add_node("understand_intent", understand_intent)
workflow.add_node("search_books", search_books_node)
workflow.add_node("analyze_preferences", analyze_preferences_node)
workflow.add_node("generate_recommendation", generate_recommendation_node)
workflow.add_node("ask_clarification", ask_clarification_node)
workflow.add_conditional_edges("analyze_preferences", route_after_analyze,
    {"ask": "ask_clarification", "recommend": "generate_recommendation"})
compiled = workflow.compile(checkpointer=MemorySaver())
```

4.4 配置文件

系统使用 `python-dotenv` 加载环境变量，支持 OpenAI 兼容 API (SiliconFlow / DeepSeek / Xiaomi MIMO 等)。配置类使用 Python `dataclass` 实现，支持多级 fallback。

```
# .env 配置文件（不硬编码 API Key，通过环境变量注入）
API_KEY=sk-your-api-key-here      # 必填: LLM API Key
API_BASE_URL=https://api.xxx.com/v1 # API Base URL
MODEL=your-model-name             # 模型名称
HOST=0.0.0.0 PORT=8000 DEBUG=true
CHROMA_PERSIST_DIR=./data/chroma   # 向量数据库目录

# Config 类 (config.py) - 支持多级 fallback
@dataclass
class Config:
    api_key: str = field(default_factory=lambda: os.getenv(
        "API_KEY",
        os.getenv("ANTHROPIC_API_KEY", os.getenv("OPENAI_API_KEY", ""))))
    api_base_url: str = field(default_factory=lambda: os.getenv(
        "API_BASE_URL", "https://api.siliconflow.cn/v1"))
    model: str = field(default_factory=lambda: os.getenv(
        "MODEL", os.getenv("ANTHROPIC_MODEL", "gpt-4o")))
```

五，测试与评估

5.1 功能测试

项目编写了 26 个自动化测试用例 (pytest + pytest-asyncio)，覆盖三层架构：

测试层级	文件	用例数	覆盖内容
API 接口测试	test_api.py	9	健康检查 / 搜索验证 / 对话 CRUD / 工作流
工具层测试	test_tools.py	8	Open Library 搜索 / 书籍详情 / 题材提取 / 偏好分析
工作流测试	test_workflow.py	9	意图识别 / 搜索节点 / 偏好分析 / 条件路由

全部 26 个测试用例通过 (pytest -v, 执行耗时 18.30s)。测试覆盖了正常路径，边界条件 (空查询, 无效 Key)，异常路径 (不存在的对话 ID 返回 404)。

```
$ pytest tests/ -v
===== test session starts =====
tests/test_api.py::test_root_returns_html PASSED [ 3%]
tests/test_api.py::test_health_check PASSED [ 7%]
tests/test_api.py::test_list_conversations_empty PASSED [ 11%]
tests/test_api.py::test_search_endpoint_validation PASSED [ 15%]
tests/test_api.py::test_search_endpoint PASSED [ 19%]
tests/test_api.py::test_chat_endpoint_validation PASSED [ 23%]
tests/test_api.py::test_get_nonexistent_conversation PASSED [ 26%]
tests/test_api.py::test_delete_nonexistent_conversation PASSED [ 30%]
tests/test_api.py::test_workflow_recommend PASSED [ 34%]
...
===== 26 passed in 18.30s =====
```

5.2 Agent 行为评估

Agent 行为评估从以下四个维度进行：

- 意图识别准确率：测试了搜索，推荐，闲聊三类意图，分类全部准确。test_understand_intent_* 全部通过。
- 工具调用正确性：当用户提出明确的书籍需求时，Agent 正确触发 search_books 工具；信息不足时，analyze_preferences_node 检测到关键词不足，触发追问而非盲目搜索。
- 多轮对话连贯性：ConversationMemory 维护对话上下文(滑动窗口 20 轮)，ChromaDB 存储用户偏好向量，确保跨轮次推荐的一致性。
- 错误处理：LLM 调用失败时返回友好提示 —— 区分认证失败(401)，频率限制(429)，超时三类错误。Open Library API 超时时优雅降级为空结果。

5.3 Demo 演示

系统启动后访问 <http://localhost:8000> 即可体验完整功能。搜索接口已验证返回真实的 Open Library 数据(如搜索 'Lord of the Rings' 返回 913 条结果)。前端支持响应式布局，移动端侧边栏自动折叠。

六，系统升级与扩展

6.1 可扩展架构

系统设计了高度可扩展的分层架构，核心模块之间通过接口解耦，每层可独立替换：

- LLM 层可替换：BaseAgent 封装了 LLM 调用接口，通过切换 API_BASE_URL 和 MODEL 即可接入不同 LLM 提供商，无需修改业务代码。已实际验证 Anthropic SDK -> OpenAI SDK 的迁移，仅修改 base_agent.py 一个文件。
- 记忆后端可替换：VectorStore 抽象了向量存储接口，当前使用 ChromaDB 嵌入式部署(零配置)，未来可平滑迁移至 Milvus / Pinecone / Weaviate 等生产级向量数据库。

- 工具层可扩展：只需在 TOOL_DEFINITIONS 中添加新的 Function 定义并实现 tool_handler 分支，即可为 Agent 增加新能力(书评分析，价格比较，借阅查询)。

6.2 下一阶段计划

阶段	计划内容	预期产出
Phase 2	接入 Google Books API，增加中文书籍覆盖率	双语书籍搜索
Phase 3	LangFuse 全链路 LLM 追踪与评估面板	LLMOps 可观测
Phase 4	用户认证 + 对话持久化(SQLite)	多用户支持
Phase 5	Docker 容器化 + docker-compose 一键启动	容器化部署
Phase 6	RAG 书评检索增强 + 多模态以图搜书	推荐质量跃升

6.3 AI 能力演进路径

- 短期(3个月)：完善书籍推荐效果，增加用户反馈机制(点赞/踩)，基于反馈优化推荐策略。引入更多书籍数据源。
- 中期(6个月)：引入 RAG(检索增强生成)，将书籍评论，书摘等非结构化数据纳入推荐依据。引入多模态能力，支持用户上传书封照片进行以图搜书。
- 长期(12个月)：构建阅读社区 Agent 网络 -- 每个用户拥有个性化阅读 Agent，Agent 之间可共享阅读品味图谱，实现社交化推荐和阅读小组智能匹配。

七，课程总结

7.1 个人收获

- SDD 思维转变：规格驱动开发让我养成了「先设计再编码」的工程习惯。在编写 4 份 Spec 文档的过程中，系统的全局面貌逐渐清晰 -- 数据模型，API 契约，Agent 行为边界在编码前就已定义完毕。后续编码几乎没有遇到架构层面的返工。这种「慢即是快」的设计理念将深刻影响我今后的工程实践。
- Agent 系统理解：通过亲手实现 Tool Use 循环，LangGraph 状态机，ChromaDB 向量记忆，我对 AI Agent 的核心运作机制 -- 「推理 -> 行动 -> 观察 -> 再推理」的 ReAct 循环 -- 有了具体而深入的理解。之前只是「使用 ChatGPT」，现在理解了 Agent 如何通过代码精确编排 LLM 的决策边界和行动能力。
- 工程闭环体验：从 SDD 规格 -> 工具开发 -> Agent 实现 -> workflow编排 -> API 封装 -> 前端展示 -> 测试 -> 可观测性，完整走通了 AI 应用从 0 到 1 的全流程，获得了宝贵的端到端工程经验。26 个自动化测试 + JSON 结构化日志 + Tracing ID 的质量保障体系，让「能跑」和「可靠」之间有了清晰的工程边界。

7.2 工程思维转变

本课程最大的收获是建立了「AI 工程化」的思维框架 -- 从三个维度完成了认知跃迁：

- 从 Prompt 到 Program: 理解了 Prompt Engineering 只是起点. 真正的 AI 应用需要通过代码精确控制 Agent 的行为边界(System Prompt), 工具调用协议(JSON Schema), 状态流转逻辑(LangGraph 条件路由).
- 从 Demo 到 Product: 一个能跑通的 Demo 和可交付的产品之间有巨大的鸿沟 -- 需要 API 设计(RESTful + SSE 流式), 错误处理(分级错误提示), 响应式 UI(移动端适配), 测试覆盖(26 用例), 日志追踪(Tracing ID)等工程实践的支撑.
- 从单点到系统: AI Agent 不是孤立的 LLM 调用, 而是 LLM + 工具 + 记忆 + 工作流 + 可观测性组成的系统工程. 六大技术要素缺一不可.

7.3 对课程的建议

- 建议增加 Agent 框架(LangGraph / CrewAI / AutoGen)的实践课时和 step-by-step 教程, 目前文档学习曲线较陡, 相关中文资料偏少.
- 建议提供统一的课程专用 LLM API Key(如一定额度的免费 tokens), 降低学生接入门槛 -- 本项目在开发过程中的 API 费用是实际的开销.
- 建议增加 Agent 评估(Eval / Benchmark)的专题讲解和实操, 如使用 LangSmith / LangFuse / Braintrust 等工具对 Agent 进行系统性评估.
- 建议课程后期安排一次 AI Agent 系统设计评审, 各小组互评架构设计和 Agent 行为设计, 促进工程思维的碰撞和交流.

总结: 「书灵」项目让我第一次以完整的工程视角看待 AI Agent 系统 -- 它不仅仅是调用大模型, 而是一个集规格设计, 工具集成, 状态管理, 记忆持久化, 可观测性于一体的系统工程. 感谢戚欣老师的指导, 这门课程让我对 AI 软件工程有了全新的认识, 也为今后从事 Agent 相关开发奠定了扎实的工程基础.